

# CubeTriage

Automated diagnosis for legacy OLAP cubes

One exported file in a proposed star schema, a quantified complexity report and a human-review workbook out.  
Every trap found, flagged and priced.

*Validated on Adventure Works 2019 Enterprise Edition —  
120 review decisions found in ~2 seconds*

Malin Jacobsson

# The problem

---

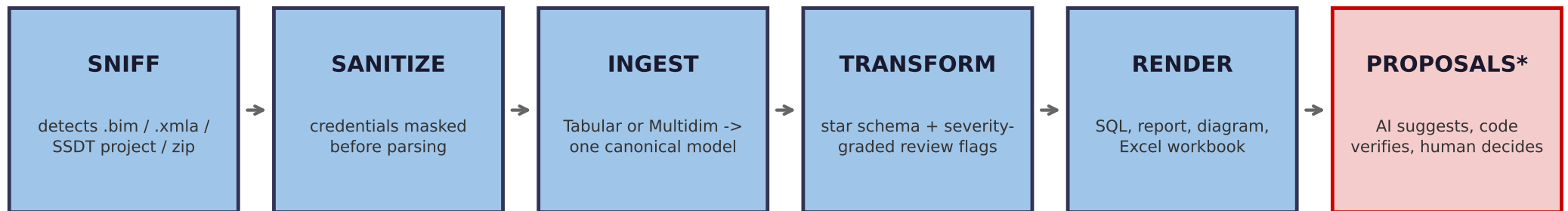
Migrating a legacy SSAS cube to a modern relational model costs a senior consultant weeks: untangling MDX, mapping role-playing dimensions, finding the KPIs and security roles nobody documented.

Before anyone can even price the migration, somebody spends days just answering: what is actually IN this cube?

**CubeTriage answers that question in seconds — from a single exported metadata file. No cube data is read.**

# How it works

---



The first four stages are deterministic — same input, same output.  
Proposals are opt-in behind a backend interface (mock / local LLM).

Outputs: \*\_canonical.json · \*\_out.json · \*\_schema.sql · \*\_load.sql ·  
\*\_validation.sql · \*\_review.md · \*\_hitl.xlsx · \*\_diagram.png ·  
\*\_decisions.json · \*\_run\_report.json

# Result on a real cube

Adventure Works 2019 Enterprise Edition (Multidimensional) —  
Microsoft's own reference cube, built to demonstrate every  
enterprise SSAS feature.

**16 dimensions · 11 facts · 79 relationships · 38 measures**

mapped automatically

**22 calculated measures in MDX**

flagged for manual translation — not ignored

**29 role-playing · 4 parent-child · 4 reference · 6 m2m bridges**

structural traps found

**31 unmigratable constructs (KPIs, actions, perspectives)**

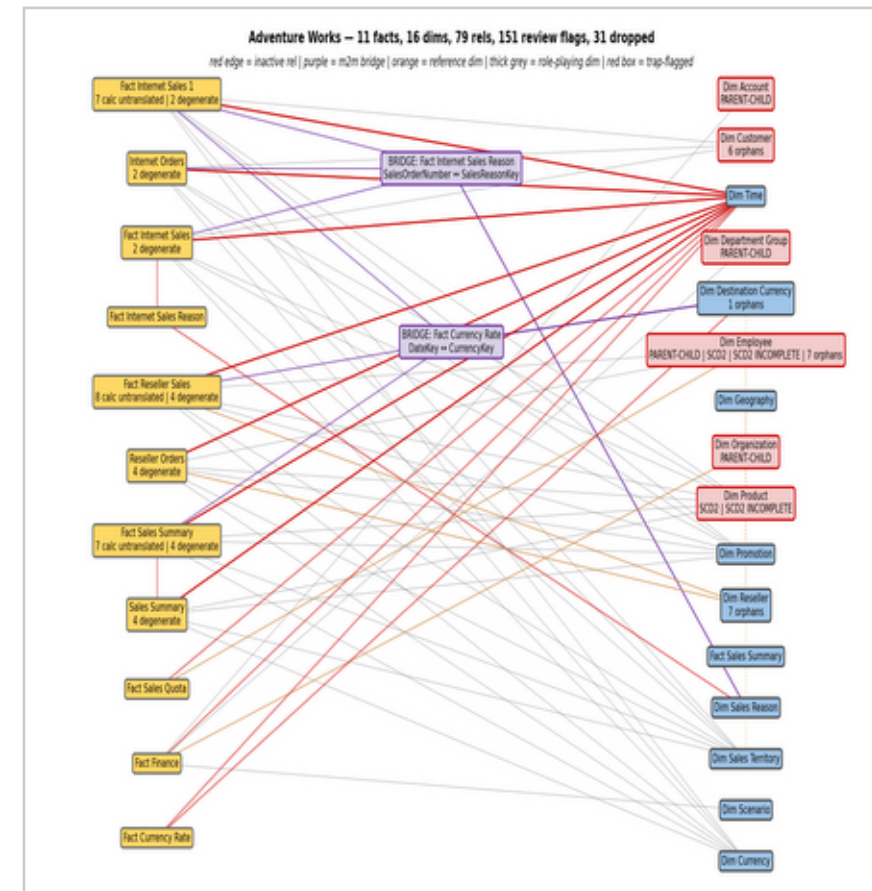
reported, never lost

**101 inferred assumptions**

each documented in its own channel

**120 human-review items**

the workbook IS the work order



Full pipeline run: ~2 seconds. Also verified on the  
Tabular AdventureWorks .bim (compat 1400) — tables classified  
purely from the relationship graph, no naming conventions needed.  
Newest: generated load SQL + a 101-check validation harness.

# The review loop

## Every warn flag and dropped item becomes a numbered

decisions.json — stable ids, survives re-runs

## Click a flag — see where the problem lives

canonical source slice + star-schema target table

## Approve · reject · override — then regenerate

decided entries persist; unapplied overrides are reported, never written silently

## Zero-dependency, offline, runs where the data lives

the Excel workbook stays as the exportable record

The screenshot displays a web application interface for managing data decisions. The top bar shows 'adventureworks' and '2 of 115 decided', with a 'Download decisions.json' button. Below this is a table with columns: 'open', 'approved', 'rejected', 'overridden', and 'all'. The 'open' column has a dropdown menu showing '113'. The table lists various decisions, each with a unique ID, a status (e.g., 'orphan\_attribute', 'parent\_child\_flattened', 'scd2\_incomplete', 'scd2\_dimension'), and a target table (e.g., 'Dim Customer.Date First Purchase', 'Dim Customer.Phone', 'Dim Customer.Email Address', 'Dim Customer.CommuteDistanceSort', 'Dim Customer.Address Line1', 'Dim Department Group', 'Dim Destination Currency.LCID', 'Dim Employee', 'Dim Employee'). Each row has an 'open' button. The right panel shows a detailed view of a decision with ID 'DB7AE3F' and status 'orphan\_attribute'. It displays the 'SOURCE' as 'MODEL/DIMENSIONS/DIM\_CUSTOMER/ATTRIBUTES/DATE\_FIRST\_PURCHASE' and the 'TARGET' as 'dbo DimCustomer.Date First Purchase'. The 'DECISION - CURRENT: OPEN' section shows three buttons: 'Approve' (green), 'Reject' (red), and 'Override' (blue). Below these is a 'Comment...' field and 'Save comment' and 'Reopen' buttons. A JSON snippet is visible in the source section: { "name": "Date First Purchase", "source\_column": "SimpleDateFirstPurchase", "source\_table": "dbo DimCustomer", "hidden": true, "orphan": true }

*Regeneration with decisions applied: ~1 second.*

# Design principles

---

## **Nothing is hidden**

Constructs the engine sees but cannot migrate are recorded in a dropped channel. A reviewer always sees the full picture.

## **Provenance is structural**

Observed facts, inferred values, dropped constructs and AI proposals are four separate channels — what the engine KNOWS vs GUESSES.

## **AI proposes — never decides**

An optional backend may suggest resolutions. Code verifiers check every proposal against the model; the human approves.

## **Runs where the data lives**

Deterministic, offline-capable, no telemetry — designed to run as a container inside the customer's own perimeter.